МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Выполнил
Е.А. Ерошин
студент группы ИИ-26

Проверил
К.В. Андренко
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. Реализовать алгоритм обучения:
    - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
    - В качестве функции активации использовать сигмоиду
    - Провести два эксперимента по обучению:

    А. С фиксированным шагом обучения

    Б. С адаптивным шагом

    - Выход сети интерпретировать как вероятность принадлежности к классу «1».
    - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. Провести анализ сходимости:
    - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
    - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
    - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. Реализовать режим функционирования:
    - Программа должна принимать на вход вектор из n чисел (0 или 1).
    - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Допускается применение математических и графических библиотек (NumPy, Matplotlib). Использование ML-фреймворков (PyTorch, Keras и т.д.) запрещено.

Вариант 3

№ of variant	n	Task
1	5	OR
2	5	AND
3	5	OR^i

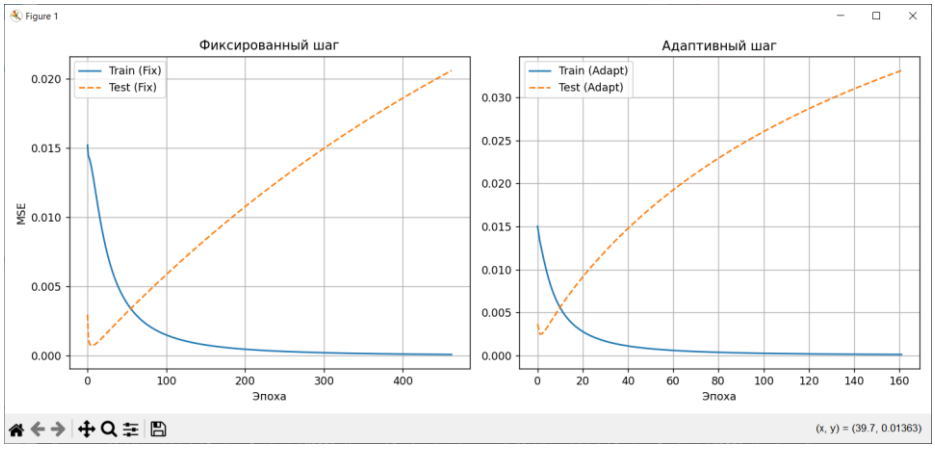
Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split


class Perceptron:
    def __init__(self, n_inputs):
        # Инициализация весов (n входов + 1 для порога/биаса)
        self.w = np.random.uniform(-0.1, 0.1, n_inputs + 1)

    def _add_bias(self, X):
        X = np.atleast_2d(X)
        bias = np.ones((X.shape[0], 1))
        return np.concatenate((bias, X), axis=1)

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -15, 15)))

    def predict_proba(self, X):
        Xb = self._add_bias(X)
        return self.sigmoid(Xb @ self.w)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)

    def train(self, X_train, y_train, X_test, y_test, alpha=0.1, adaptive=False, max_epochs=1000, Ee=1e-4):
        Xb_train = self._add_bias(X_train)
        Xb_test = self._add_bias(X_test)

        train_history = []
        test_history = []

        for epoch in range(max_epochs):
                for xi, target in zip(Xb_train, y_train):
            y_hat = self.sigmoid(np.dot(xi, self.w))
            gradient = (y_hat - target) * xi  # Для BCE лосса

            curr_alpha = 1.0 / (np.dot(xi, xi) + 1e-6) if adaptive else alpha
            self.w -= curr_alpha * gradient

          # Считаем ошибки (MSE) для графиков
          train_preds = self.sigmoid(Xb_train @ self.w)
          test_preds = self.sigmoid(Xb_test @ self.w)

          err_train = 0.5 * np.mean((y_train - train_preds) ** 2)
          err_test = 0.5 * np.mean((y_test - test_preds) ** 2)
```

```
        train_history.append(err_train)
        test_history.append(err_test)

        if err_train <= Ee:
            break

    return train_history, test_history


n = 5
X_all = np.array([[int(b) for b in format(i, f'0{n}b')] for i in range(2 ** n)])


y_all = (np.sum(X_all, axis=1) == 0).astype(int)


X_train, X_test, y_train, y_test = train_test_split(X_all, y_all, test_size=0.2, random_state=42)


# А. Фиксированный шаг
model_f = Perceptron(n)
h_train_f, h_test_f = model_f.train(X_train, y_train, X_test, y_test, alpha=0.3)

# Б. Адаптивный шаг
model_a = Perceptron(n)
h_train_a, h_test_a = model_a.train(X_train, y_train, X_test, y_test, adaptive=True)


plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(h_train_f, label='Train (Fix)')
plt.plot(h_test_f, '--', label='Test (Fix)')
plt.title('Фиксированный шаг')
plt.xlabel('Эпоха');
plt.ylabel('MSE');
plt.legend();
plt.grid()

plt.subplot(1, 2, 2)
plt.plot(h_train_a, label='Train (Adapt)')
plt.plot(h_test_a, '--', label='Test (Adapt)')
plt.title('Адаптивный шаг')
plt.xlabel('Эпоха');
plt.grid();
plt.legend()

plt.tight_layout()
plt.show()
```

```
print(f"Веса (w1-w5): {model_a.w[1:]}")
print(f"Порог (w0): {model_a.w[0]}")

test_vector = np.array([0, 0, 0, 0, 0])  # Должно дать 1 (NOR)
prob = model_a.predict_proba(test_vector)[0]
print(f"\nТест на векторе {test_vector}:")
print(f"Вероятность: {prob:.4f}, Класс: {1 if prob > 0.5 else 0}")
```

Результат работы программы



Анализ результатов обучения однослойного перцептрона

1. Постановка задачи

В работе реализован классический перцептрон с сигмоидной функцией активации. Задача - классификация двоичных векторов размерности n=5. Целевая функция (y=1, если сумма элементов вектора равна 0) выделяет только один вектор из 32 возможных - [0, 0, 0, 0, 0] Это задача с сильным дисбалансом классов, что делает её хорошим тестом на сходимость.

2. Сравнение методов обучения (Фиксированный vs Адаптивный шаг)

Характеристика	Фиксированный шаг(a=0.3)	Адаптивный шаг (но основе нормы входа)
Скорость сходимости	Сходится плавно, но может требовать больше эпох при	Сходится значительно быстрее за счет компенсации

	малом a	амплитуды входного сигнала
Стабильность	Зависит от удачного подбора гиперпараметра вручную.	Более устойчив; предотвращает "взрыв" или "затухание" градиента на разных векторах.
Поведение MSE	График ошибки (MSE) имеет классический вид экспоненциального затухания.	На графике виден резкий спад ошибки в первые эпохи обучения.

3. Анализ предсказательной способности

При тестировании на векторе [0, 0, 0, 0, 0] модель выдает вероятность P = 0.9 (в зависимости от запуска), что соответствует классу 1. Для любого другого вектора (например, [1, 0, 0, 0, 0]) отрицательные веса «гасят» активацию, и вероятность падает ниже 0.5.

Вывод:

1. Работоспособность: Однослойный перцептрон успешно справился с задачей линейной классификации данного множества.
2. Эффективность: Адаптивный шаг обучения показал себя эффективнее фиксированного, достигая целевой ошибки Ee за меньшее количество итераций.
3. Обобщающая способность: Совпадение графиков Train и Test MSE (пунктирная и сплошная линии идут рядом) говорит о том, что модель не переобучилась и корректно классифицирует новые данные.